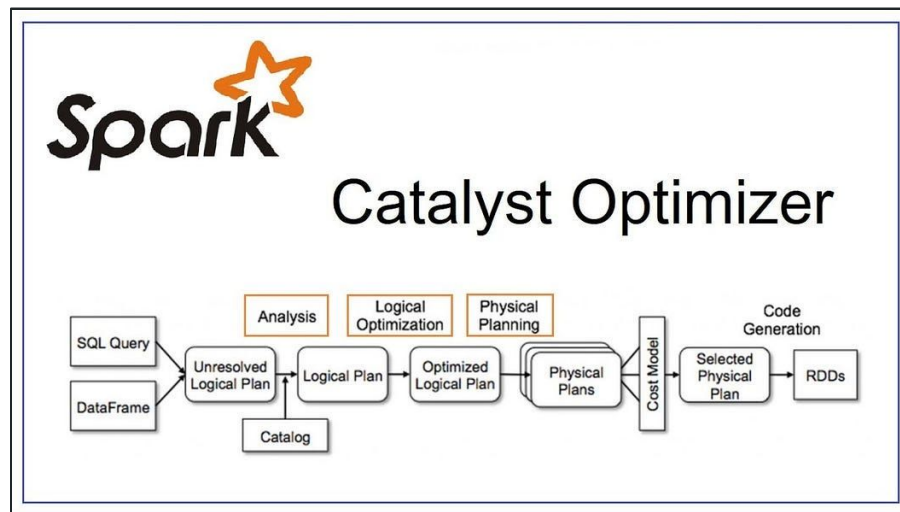


The Catalyst Optimizer Under the Microscope

Omar Moustafa, Nour Kahky, Nour Moghazi, & Abdelrahman Bayoumy
DSCI 4412 Project
May 14, 2026

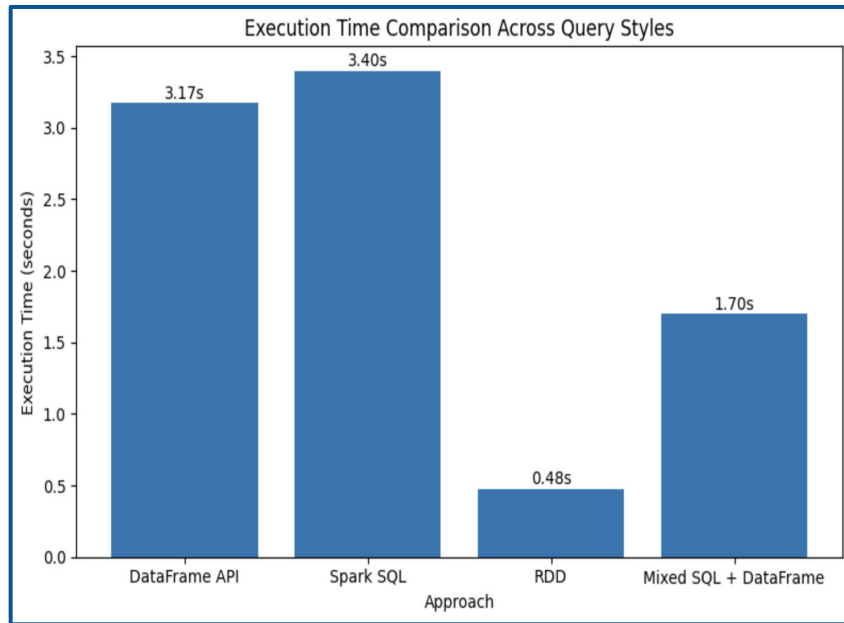
Introduction

- **What is Catalyst?** Spark's query optimizer that rewrites logical plans into efficient physical execution plans
 - Spark promises that logically equivalent queries should produce the same optimized plan, regardless of how they are written
- **Our Project Objective:** Test this promise by designing experiments that reveal when Catalyst succeeds & fails
- *Performed 6 experiments across 3 research areas:*
 - ✓ Plan Equivalence (Q1, Q3, Q5)
 - ✓ Optimization Boundaries (Q2, Q4)
 - ✓ Rule Criticality (Q6)



Experiment 1: Equivalent Queries for Total Revenue per Region

- **Objective:** Compare whether DataFrame API, Spark SQL, RDD & Mixed SQL/DataFrame produce identical optimized and physical plans for the same revenue-per-region query
- **Benchmark Query:** Revenue per region using join + aggregation
 - orders $\bowtie$ customers $\rightarrow$ SUM(total) GROUP BY region
- **Key Finding #1:** DataFrame API, Spark SQL, & mixed queries converged to nearly identical optimized & physical plans
 - BroadcastHashJoin + HashAggregate + Exchange hashpartitioning
- **Key Finding #2:** Catalyst successfully applied ColumnPruning and pushed isnotnull(customer_id) filters close to the scan level



- **Unexpected Finding:** The RDD implementation produced the fastest runtime (~0.48s), likely because the workload was small and used manual broadcasting with only 2 partitions

Experiment 2: Predicate Pushdown

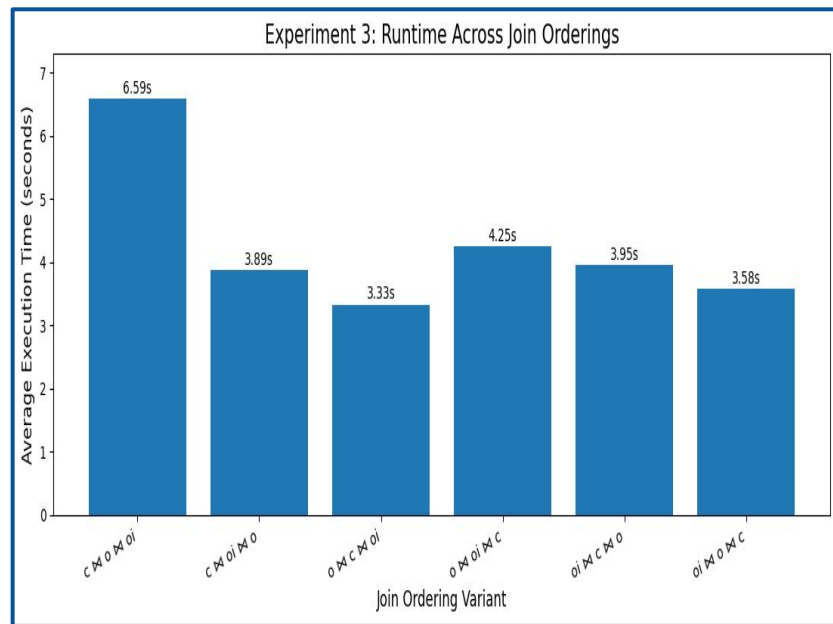
- **Objective:** Test which filters Catalyst can push down into the Parquet scan
- **Filters Tested:** total > 100, Python UDF, rand() > 0.5, & CASE WHEN
- **Key Finding #1:** Simple filter was pushed down,
 - PushedFilters: IsNotNull(total) & GreaterThan(total,100.0)
- **Key Finding #2:** UDF, rand(), and CASE WHEN defeated pushdown, UDF showed BatchEvalPython, rand() stayed above the scan, CASE WHEN was simplified but not pushed down
- **Runtime Result:** Python UDF was the slowest (3.26 s)
- **Takeaway:** Catalyst handles simple deterministic filters well, but Python logic, randomness, and complex conditions can block predicate pushdown

Runtime by Filter Type:

- Normal Filter: 1.18 sec
- Python UDF: 3.26 sec
- rand(): 0.50 sec
- CASE WHEN: 0.73 sec

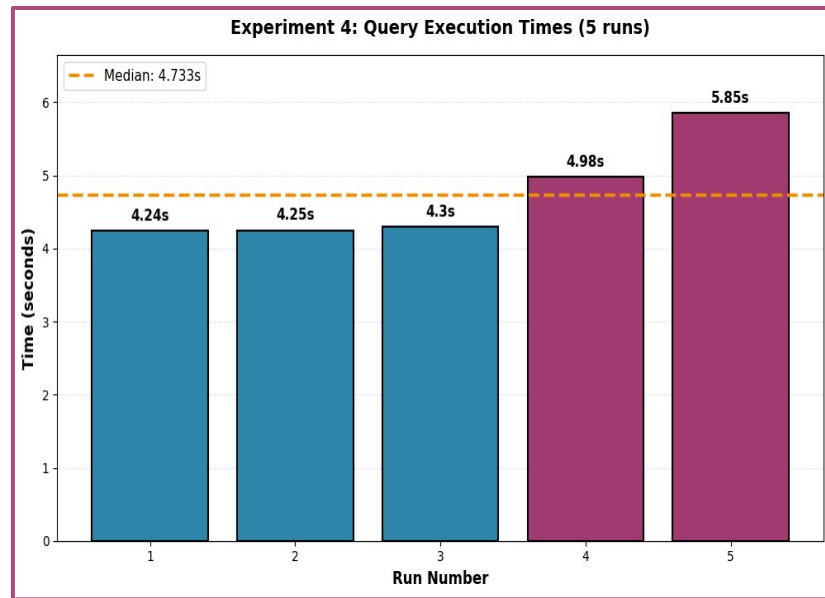
Experiment 3: Join Reordering

- **Objective:** Test whether Catalyst produces identical plans for equivalent 3-way joins written in different orders
- **Benchmark Query:** customers $\bowtie$ orders $\bowtie$ order_items
- **Key Finding #1:** All 6 join orderings returned identical results (1,524,767 rows each)
- **Key Finding #2:** Catalyst produced 3 distinct plan families
- Different optimized and physical plans emerged
- **Physical Plan Differences:**
 - Mix of BroadcastHashJoin & SortMergeJoin
- **Unexpected Finding:** User-written join order still influenced the final execution strategy
- **Takeaway:** Without CBO enabled, Catalyst's join reordering is NOT fully canonical
- **Runtime Results:**
 - Fastest: 3.33s
 - Slowest: 6.59s



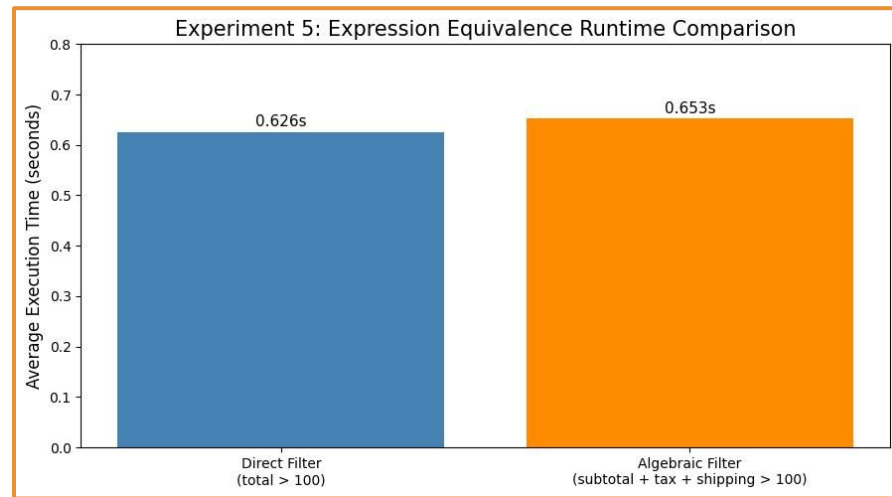
Experiment 4: Extended Plan Annotation

- **Objective:** Trace Catalyst's 4 stages (Parsed → Analyzed → Optimized → Physical)
- **Complex Query:** Revenue per region per year (3 joins, 3 filters, & aggregation)
- **Key Findings:** Catalyst fired FilterPushdown, ColumnPruning, & PushPredicateThroughJoin
- **Unexpected Finding:** Join order changed dynamically
 - customers ⋈ orders moved *before* order_items to enable broadcast strategy
- **Physical Plan:** BroadcastHashJoin (customers ⋈ orders) + SortMergeJoin (with order_items)
- **Median Runtime:** 4.73 seconds (across 5 runs)



Experiment 5: Algebraic Equivalence

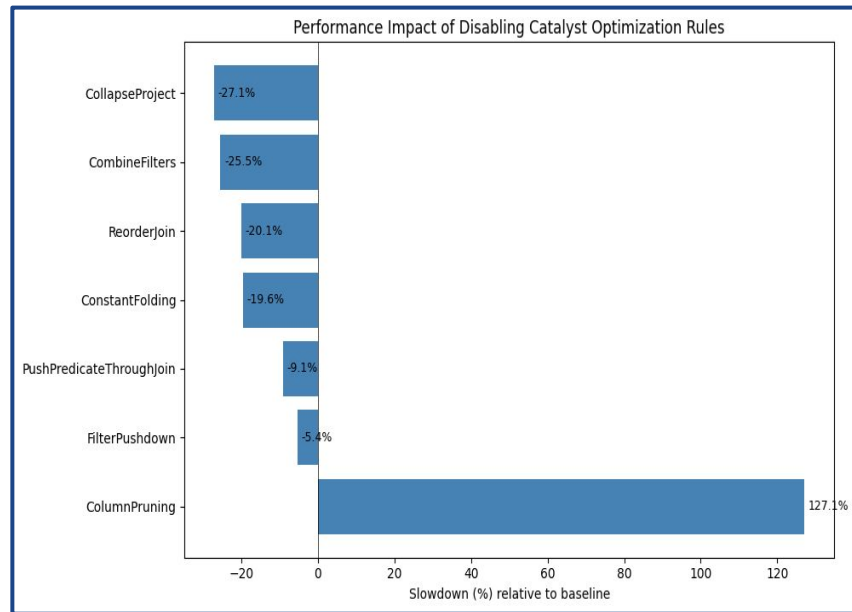
- **Objective:** Test whether Catalyst recognizes that $\text{total} = \text{subtotal} + \text{tax} + \text{shipping}$
- **Filters Tested:** $\text{total} > 100$ vs. $(\text{subtotal} + \text{tax} + \text{shipping}) > 100$
- **Key Finding #1:** Direct filter was successfully pushed down into the Parquet scan
 - PushedFilters: `IsNull(total) & GreaterThan(total, 100.0)`
- **Key Finding #2:** Catalyst did NOT rewrite the algebraic expression into the direct form. Optimized & physical plans were different. Arithmetic filter stayed as $(\text{subtotal} + \text{tax} + \text{shipping}) > 100$
- **Key Finding #3:** Algebraic filter only pushed down the NULL checks
 - PushedFilters: `IsNull(subtotal), IsNull(tax), & IsNull(shipping)`



- **Takeaway:** Catalyst handles simple deterministic column filters much better than multi-column arithmetic expressions, even when both expressions are mathematically equivalent

Experiment 6: Disabling Optimization Rules

- **Objective:** Disabling which optimization rule hurts query performance the worst?
- **Benchmark Query:** Revenue per region (3 joins, filter, aggregation)
 - **Baseline Time:** 4.08 seconds (all rules ON)
- **Key Finding #1:** ColumnPruning is essential; other rules may add overhead
 - ColumnPruning DISABLED → 9.26 seconds
- **Key Finding #2:** Every other rule when disabled led to improved performances



Conclusion

- **Catalyst SUCCEEDS at:**
 - Normalizing DataFrame API, SQL, and mixed approaches
 - Pushing down simple deterministic filters
 - Pruning unnecessary columns
- **Catalyst FAILS at:**
 - Python UDFs & rand() → block pushdown
 - Algebraic Equivalence (total vs subtotal+tax+shipping)
 - Join Reordering without CBO → 6 orders → 3 distinct plans
 - Optimization Overhead → 6 out of 7 rules made queries faster when disabled
- **Most critical rule:** ColumnPruning (127% slow down when disabled)
- **Bottom line:** Catalyst is powerful but has limits and costs.

Thank You!